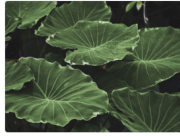


WE LIKE TO
SHOP



Grow Your Own Spy Kit
★★★★☆ \$36.07

[View details](#)



Caution Sign
★★★★★ \$85.86

[View details](#)



Packaway Carport
★★★☆☆ \$31.14

[View details](#)



Dancing In The Dark
★☆☆☆☆ \$97.90

[View details](#)



Actividad 08

File path traversal, traversal sequences blocked with absolute path bypass

Guerrero Hurtado Lizbeth

Marco Teórico

¿Qué es File Path Traversal?

Es una vulnerabilidad de control de acceso que permite a un atacante leer archivos arbitrarios alojados en el servidor que ejecuta la aplicación web. Ocurre cuando el backend toma entradas provistas por el usuario y las utiliza para construir rutas del sistema de archivos sin aplicar una validación o sanitización estricta.

Traversal Sequences Blocked with Absolute Path Bypass

Muchas aplicaciones intentan defenderse filtrando o bloqueando secuencias de navegación relativa como ../ (secuencias dot-dot-slash). Sin embargo, si el código del servidor simplemente toma el parámetro recibido y lo pasa de forma directa a una API de lectura de archivos (como `file_get_contents` en PHP o `FileInputStream` en Java) sin verificar si la ruta es relativa o absoluta, el filtro relativo resulta inútil. En este escenario, enviar una ruta absoluta directa (ej. `/etc/passwd`) omite el mecanismo de inspección por completo.

Mecanismo de Explotación

El fallo reside en que la aplicación asume que el usuario sólo enviará el nombre de un archivo (como `imagen.jpg`) para concatenarlo a un directorio base (ej. `/var/www/images/`). Si el desarrollador no valida que la entrada sea estrictamente un nombre de archivo, el sistema operativo interpreta la ruta absoluta `/etc/passwd` desde la raíz del sistema (`/`), ignorando el prefijo del directorio base o sobrescribiendo la resolución lógica de la ruta.

Punto Clave de Seguridad

El filtrado de caracteres o patrones específicos (listas negras) es una defensa débil e incompleta. Bloquear un vector de ataque específico (../) no garantiza que la función subyacente de manejo de archivos sea segura si sigue aceptando rutas absolutas o codificaciones alternativas.

Impacto en la Tríada CIA

Confidencialidad (Alto): Compromiso directo. Permite la lectura no autorizada de archivos sensibles del sistema operativo (`/etc/passwd`, `/etc/shadow`), credenciales en archivos de entorno (`.env`), claves privadas SSH y código fuente.

Integridad (Bajo/Medio): Aunque este laboratorio se enfoca en lectura, en escenarios con

permisos mal configurados de escritura podría permitir la modificación de archivos.

Disponibilidad (Medio): La lectura de archivos de dispositivos especiales (como `/dev/urandom` o `/dev/zero`) puede causar un consumo masivo de memoria y CPU, derivando en una Denegación de Servicio (DoS).

Procedimiento de Explotación (Metodología en Burp Suite)

Reconocimiento e Intercepción:

- Abrir el navegador integrado de Burp Suite y acceder a la aplicación del laboratorio.
- Navegar por la página web y seleccionar cualquier producto para forzar la carga de sus imágenes.
- Activar Burp Proxy para capturar el tráfico HTTP generado.

Identificación del Endpoint Vulnerable:

- Ir a la pestaña Proxy > HTTP history.
- Localizar la petición GET encargada de servir recursos multimedia, identificada por el endpoint:
- GET /image?filename=29.jpg HTTP/1.1

Envío al Módulo Repeater:

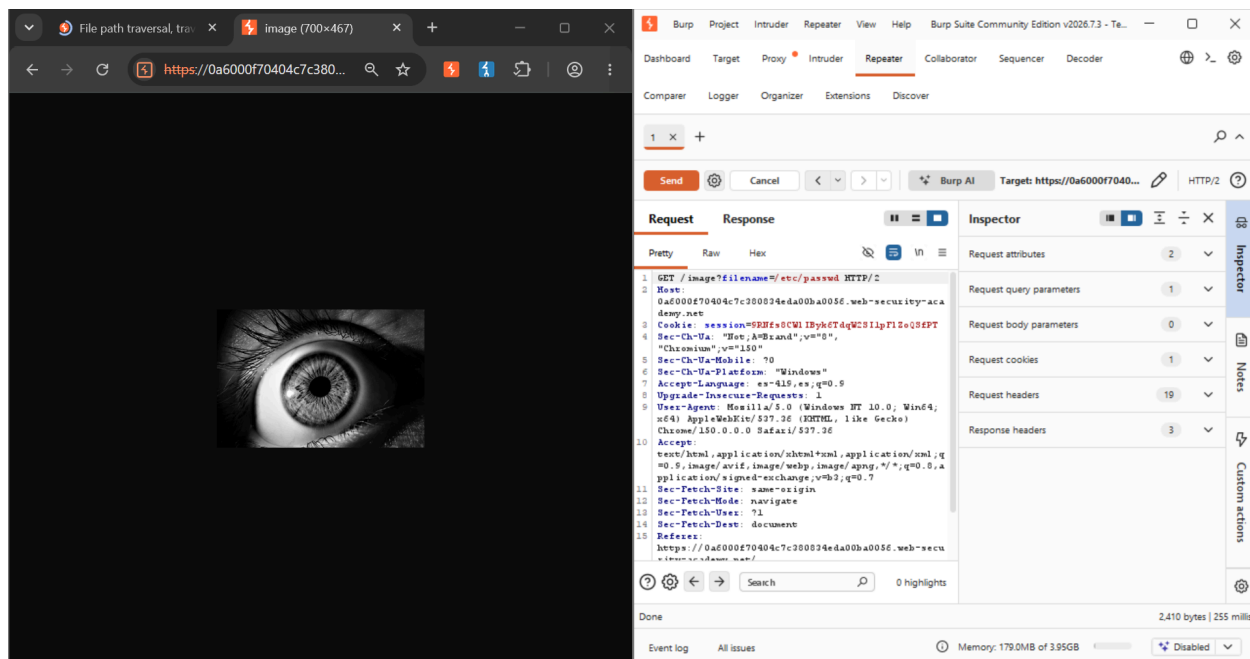
- Hacer clic derecho sobre la petición identificada y seleccionar Send to Repeater (o presionar Ctrl + R).

Inyección del Payload (Absolute Path Bypass):

- En la pestaña Repeater, modificar el parámetro filename. En lugar de utilizar secuencias relativas (`../../etc/passwd`), ingresar directamente la ruta absoluta del archivo objetivo:
- GET /image?filename=/etc/passwd HTTP/1.1

Ejecución y Confirmación:

- Hacer clic en el botón Send.
- Verificar que la respuesta del servidor sea un código de estado HTTP/2 200 OK.
- Confirmar en el cuerpo de la respuesta (Response Body) la presencia de la estructura del archivo `/etc/passwd` con la lista de usuarios del sistema.
- Refrescar la página principal para validar el banner de laboratorio resuelto (Lab Solved).



Estrategias de Mitigación Recomendadas

Evitar la Entrada de Rutas por el Usuario:

La solución ideal es no pasar nombres ni rutas de archivos directamente a través de parámetros URL. Se deben utilizar identificadores indirectos mapeados en una base de datos o un índice numérico (ej. /image?id=105).

Sanitización con Listas Blancas (Whitelist):

Si es estrictamente necesario recibir nombres de archivo, validar mediante expresiones regulares que la entrada contenga únicamente caracteres alfanuméricos y extensiones permitidas, rechazando cualquier diagonal / o \:

```
^[a-zA-Z0-9_-]+\.(jpg|png|gif)$
```

Verificación de Ruta Canónica (Canonicalization Check):

Antes de procesar el archivo, resolver su ruta canónica en el servidor y validar que comience estrictamente dentro del directorio base destinado para las imágenes:

Ejemplo en Java

```
File file = new File(BASE_DIRECTORY, userInput);  
  
if (!file.getCanonicalPath().startsWith(BASE_DIRECTORY)) {  
    throw new SecurityException("Acceso no autorizado: Intento de Path Traversal");  
}
```

Principio de Menor Privilegio:

Configurar el usuario del servicio web (ej. www-data) con permisos estrictamente limitados en el sistema operativo, impidiendo la lectura de directorios fuera del directorio raíz de la aplicación web (webroot).

Conclusiones y Aprendizaje

- Insuficiencia de las Listas Negras: Este laboratorio demuestra que intentar bloquear secuencias específicas de ataque (como ../) es una estrategia defensiva defectuosa, ya que los atacantes pueden utilizar rutas absolutas u otras variantes para saltarse el filtro.
- Importancia de la Validación Estructural: La seguridad en el manejo de archivos no debe depender de limpiar la entrada del usuario, sino de validar que la entrada coincida exactamente con lo esperado (lista blanca) o verificar programáticamente que la ruta resultante no escape del directorio permitido.
- Comprensión del Sistema Operativo Subyacente: Comprender cómo el sistema operativo resuelve las rutas de archivos (tanto relativas como absolutas) es fundamental para diseñar defensas robustas y para identificar vectores de evasión durante una auditoría de seguridad.